

Mini-project 1 in Advanced Machine Learning (02460)

by Muhammad Hussain Rashid Al-Takmaji, Ivan Musalyk, Dhruva Gajanan Sakhare & Nguyen Hoan Nhi Tran
on 05 March 2026

Part 1. We train VAEs with latent dimension 2 and batch size 256. The standard Gaussian and Mixture of Gaussian (MoG) Tomczak (2024) priors are trained for 100 epochs, while the flow-based prior is trained for 50 epochs. Both encoder and decoder employ a fully connected neural networks.

The **standard Gaussian prior** produces a latent space where most samples concentrate near the origin, leading to strong overlap between digit classes and reconstruction ambiguities (e.g., digits 4 and 9). Using a **MoG prior (10 components)** introduces multiple modes, encouraging the encoder to map datapoints to different regions and yielding clearer class structure, although some overlap remains near the center. In contrast, the **flow-based prior** learns a flexible non-Gaussian distribution by warping the latent space; while its contours do not perfectly match the aggregated posterior, this reshaping helps separate morphologically similar digits more effectively.

Table 1 shows that the MoG prior achieves the highest ELBO, then the flow-based prior, finally the standard normal prior. The improvement of MoG likely comes from its flexibility: multiple mixture components can better approximate the multimodal aggregate posterior, reducing KL divergence. Flow-based shows a mismatch between its aggregated posterior and prior. However, flow-based can discriminate morphologically similar digits like 4 and 9 better than standard Gaussian, as the flow-based prior reshapes the latent geometry, allowing the encoder to place ambiguous digits (like 4 and 9) in better-separated regions. Therefore, a high ELBO can be attributed to a high KL regularization term. In terms of stability, the flow-based prior has the lowest standard deviation, while MoG shows higher variability. Overall, more expressive priors can improve ELBO, but their effectiveness depends on how well the model exploits this flexibility during training.



Figure 1: Contour plots of VAE with: (Left) Standard Gaussian prior. (Middle) Mixture of Gaussian prior. (Right) Flow-based prior

Table 1: Report of ELBO over 10 training runs

Method	Mean	Standard deviation
Standard normal	-133.8738	0.6948
MoG	-133.3765	0.9544
Flow-based	-133.6960	0.3759

Part 2. The standard DDPM employs a convolu-

tional U-Net architecture with SiLU activations. The timestep t is spatially expanded and concatenated to the input as an additional image channel. The latent DDPM employs a fully connected β -VAE Higgins et al. (2017) with a 20-dimensional latent space, whose decoder uses a tanh activation function to have output within range $[-1; 1]$, which is then transformed back to $[0; 1]$ for visualization. For visualizing the latent space, we adopt a 2-dimension β -VAE.

Table 2: FID and sampling performance over 10 runs (2000 samples).

Method	β	FID	Samples/s
MoG	–	18.78 ± 1.11	3477.52
DDPM	–	13.59 ± 0.99	27.02
Latent DDPM	10^{-6}	12.70 ± 0.56	1991.54
Latent DDPM	0.3	14.21 ± 0.91	1973.29
Latent DDPM	1.0	36.17 ± 1.00	1945.55

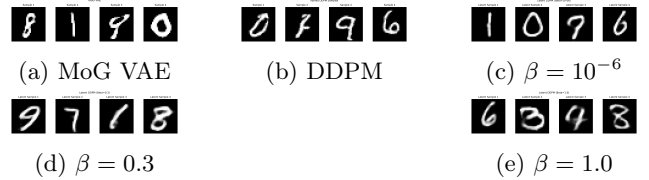


Figure 2: Generated samples from different models.

The Latent DDPM with $\beta = 10^{-6}$ achieves the best performance, obtaining the lowest FID while maintaining high sampling efficiency. The U-Net DDPM achieves a comparable FID but is approximately 74 times slower due to the iterative diffusion process operating directly in pixel space. In contrast, the MoG VAE achieves the highest sampling speed since it does not rely on diffusion, though this results in lower image quality with an FID.

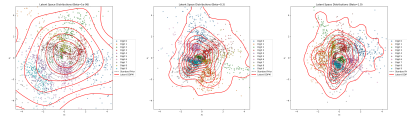


Figure 3: Prior, learned latent DDPM distribution, and aggregate posterior for $\beta = 10^{-6}$ (left), $\beta = 0.3$ (middle), and $\beta = 1.0$ (right).

Comparing Latent DDPMs trained with $\beta = 10^{-6}$ and $\beta = 1.0$ demonstrates the impact of latent regularization. With $\beta = 1.0$, the latent distribution is strongly constrained to match a standard Gaussian prior, which reduces reconstruction quality (high FID). When $\beta = 10^{-6}$, the aggregate posterior can deviate from the prior to prioritize reconstruction quality. The DDPM subsequently learns this more complex aggregate posterior distribution as seen in Figure 3, allowing the decoder to generate higher-quality images than would be possible by sampling directly from the standard VAE prior.

Code snippets

Central implementation of Flow-based.

```

1  def forward(self, z):
2      zm = z * self.mask
3      s = self.scale_net(zm) * (1 - self.mask)
4      t = self.shift_net(zm) * (1 - self.mask)
5      z = zm + (1 - self.mask) * (z * torch.exp(s) + t)
6      log_det = (s * (1 - self.mask)).sum(-1)
7      return z, log_det
8  def log_prob(self, z):
9      log_det, u = torch.zeros(z.size(0), device=z.device), z
10     for l in reversed(self.layers):
11         m = l.mask.to(z.device)
12         um = u * m
13         s = l.scale_net(um) * (1 - m)
14         t = l.shift_net(um) * (1 - m)
15         u = um + (1 - m) * ((u - t) * torch.exp(-s))
16         log_det -= (s * (1 - m)).sum(-1)
17     return self.base.log_prob(u) + log_det

```

ELBO evaluation of non-Gaussian prior:

```

1  class MoGPrior(nn.Module): # Source: J. M. Tomczak. Deep Generative Modeling. Springer, 2024.
2      def __init__(self, M, K):
3          super().__init__()
4          self.mu = nn.Parameter(torch.randn(K, M)*2)
5          self.log_std = nn.Parameter(torch.zeros(K, M)-0.5)
6          self.logits = nn.Parameter(torch.zeros(K))
7      def forward(self):
8          mix = td.Categorical(logits=self.logits)
9          comp = td.Independent(td.Normal(self.mu, torch.exp(self.log_std)),1)
10         return td.MixtureSameFamily(mix, comp)
11 #Flow prior's log_prob() is defined above
12 def elbo(self, x):
13     q = self.encoder(x); z = q.rsample(); pz = self.prior()
14     log_px = self.decoder(z).log_prob(x)
15     kl = q.log_prob(z) - pz.log_prob(z)
16     return torch.mean(log_px - kl)

```

DDPM ELBO

```

1  def negative_elbo(self, x):
2      t = torch.randint(0, self.T, (x.shape[0],), device=x.device)
3      epsilon = torch.randn_like(x)
4      a_bar = self.alpha_cumprod[t].view(-1, 1)
5      x_t = torch.sqrt(a_bar) * x + torch.sqrt(1 - a_bar) * epsilon
6      epsilon_theta = self.network(x_t, t.float().view(-1, 1) / self.T)
7      return F.mse_loss(epsilon_theta, epsilon, reduction='none').sum(dim=-1)

```

DDPM Sampling

```

1  @torch.no_grad()
2  def sample(self, shape):
3      x = torch.randn(shape, device=self.beta.device)
4      for t in reversed(range(self.T)):
5          t_in = torch.full((shape[0],1),t,device=x.device)/self.T
6          eps = self.network(x, t_in)
7          a, a_bar, b = self.alpha[t], self.alpha_cumprod[t], self.beta[t]
8          mean = (1/torch.sqrt(a))*(x-(1-a)/torch.sqrt(1-a_bar)*eps)
9          x = mean + torch.sqrt(b)*torch.randn_like(x) if t>0 else mean
10     return x

```

Latent DDPM training and sampling

```

1  b_vae.eval(); latent_ddpm.train()
2  for x,_ in dataloader:
3      x = x.to(device)
4      with torch.no_grad():
5          mu,logvar = b_vae.encode(x)
6          z = b_vae.reparameterize(mu,logvar)
7          optimizer_ddpm.zero_grad()
8          loss = latent_ddpm.loss(z)
9          loss.backward(); optimizer_ddpm.step()
10 #Sampling algorithm
11 with torch.no_grad():
12     z = latent_ddpm.sample((num_samples, latent_dim))
13     generated_images = b_vae.decode(z)

```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: [yes]

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Gemini and ChatGPT

Describe how the tools were used:

What did you use the tool(s) for? We used the tools to aid us reviewing and refactoring our code and aid in writing the report in latex.

At what stage(s) of the process did you use the tool(s)? We used the tools in the review process, after developing the main code.

We used the tools during the writing of the report for latex formatting.

How did you use or incorporate the generated output? We used the generated output to aid in writing of the report and organizing our code.

References

- I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017.
- J. M. Tomczak. *Deep Generative Modeling*. Springer, 2024. URL <https://link.springer.com/book/10.1007/978-3-031-64087-2>.